

Social Support Workflow Automation

Solution Summary

Suvojit Hore | github.com/suvojith/social_support_automation

1. Introduction and Solution Overview

A government social security department provides financial support and economic enablement to applicants in need. The current process takes 5 to 20 working days: applicant details are re-typed from scanned documents, inconsistencies between documents are hunted down manually, applications pass through several departmental review rounds, and the final judgment varies from reviewer to reviewer.

This prototype automates that workflow end to end. An applicant (or caseworker) submits an interactive application form together with five supporting documents — bank statement, credit report, Emirates ID, resume, and an assets/liabilities spreadsheet. A pipeline of six LangGraph agents extracts the data, cross-checks it for inconsistencies, scores eligibility with an explainable machine-learning model, and returns one of three recommendations: **approve**, **soft decline** (no financial support, but economic enablement is still offered), or **route to a human caseworker** when the case cannot be decided confidently. Every recommendation carries a SHAP explanation, a rationale written from the case's own figures, and an enablement plan (upskilling, job matching, or career counselling).

All models run locally through Ollama — no applicant data leaves the machine — and the whole stack starts with a single command (`bash setup.sh`), which installs Docker and Ollama when missing, pulls the models, starts roughly ten containers, seeds four databases, and trains the classifier.

Key measured outcomes of the prototype:

Measure	Result
End-to-end decision time	~25 seconds per application (M3 Max, local models)
Routing correctness on the 15 seeded test citizens	15/15 — every case reaches its designed outcome
Eligibility classifier (5-fold CV, vs. logistic-regression baseline)	F1 0.831 vs 0.790 · ROC-AUC 0.861 vs 0.829
Document extraction accuracy vs. ground truth	100% on parsed PDF/Excel fields · 93% on vision-OCR fields
Chat quality (LLM-as-judge, 1–5)	Groundedness 5.0 · rationale consistency 5.0

2. High-Level Architecture

The system is organised in five layers: a bilingual Streamlit front-end, a FastAPI serving layer, a LangGraph orchestration layer containing the six agents, a data layer of four purpose-built stores, and a local model layer served by Ollama. Langfuse traces every agent and model call end to end.

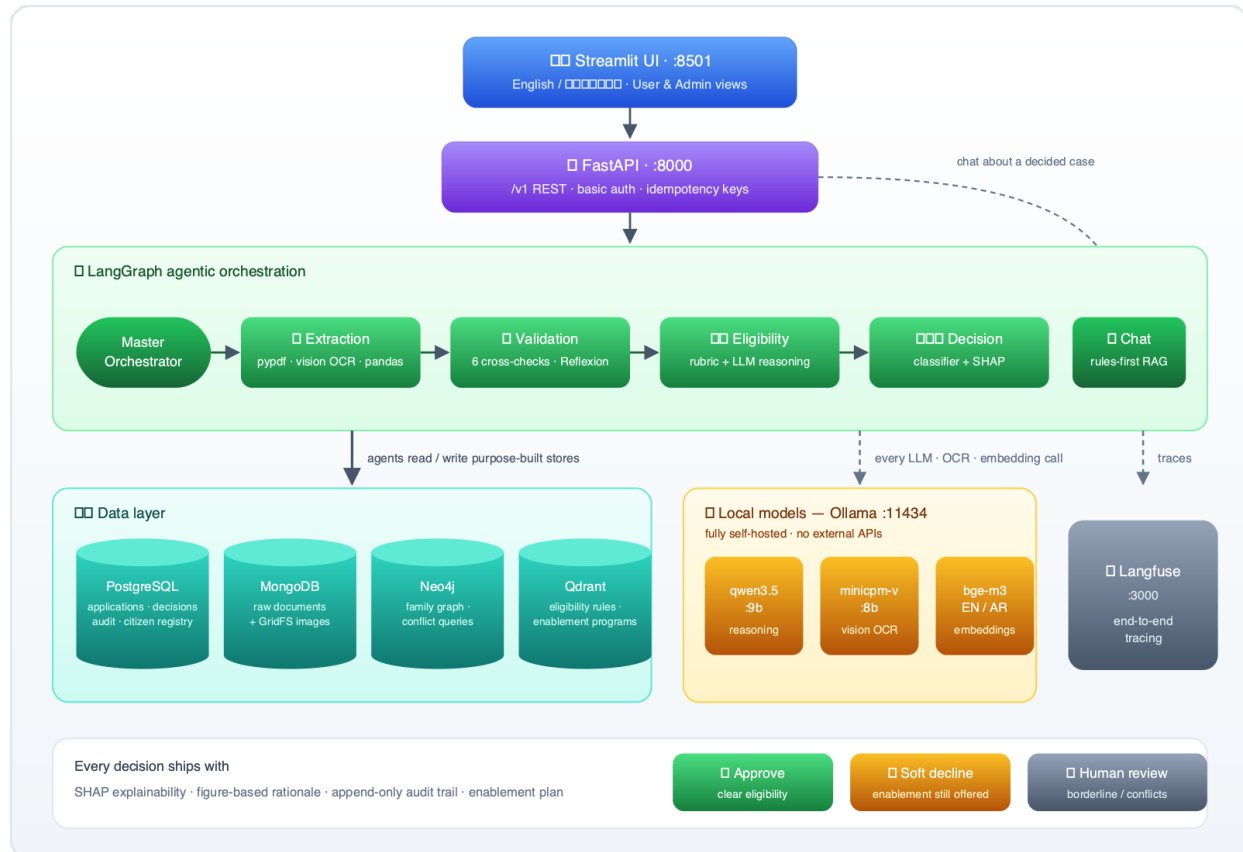


Figure 1 — component architecture. Solid arrows are the request path; dashed arrows are model and tracing calls.

2.1 Data flow

1. A submission enters through the Streamlit UI (or directly through the REST API) as a form plus five base64-encoded documents. An optional idempotency key protects against duplicate submissions.
2. Fast API persists the application record in PostgreSQL, records the idempotency fingerprint, writes an audit entry, and invokes the LangGraph workflow.
3. The extraction agent routes each document to a type-specific extractor — pypdf for PDFs, a vision LLM (with Tesseract backstop) for the Emirates ID image, pandas for the spreadsheet — and mirrors the family members it finds into Neo4j, one node per person per source document.
4. The validation agent runs six deterministic cross-checks (document completeness, identity against the ID-card OCR, address, bank-vs-bureau income, employment against the resume, and a Neo4j query for family date-of-birth conflicts) plus an LLM pass that critiques its own findings before returning them (Reflexion).
5. The eligibility agent assembles the ten-feature dictionary and applies the rubric: per-capita income bands and net-worth bands, with a $\pm 10\%$ borderline zone around each cutoff.

6. The decision agent combines the rubric signal, the validation flags, and the classifier probability into a recommendation with SHAP values; borderline or conflicted cases branch to the human-review queue instead of being force-decided.
7. The decision, its figure-based rationale, the validation flags, and the SHAP values are persisted in PostgreSQL; extracted data is PII-masked and stored in MongoDB; the audit trail is appended.
8. The chat agent answers caseworker questions about any decided application, grounded in the persisted decision record and in enablement programs retrieved from Qdrant.

2.2 Decision routing

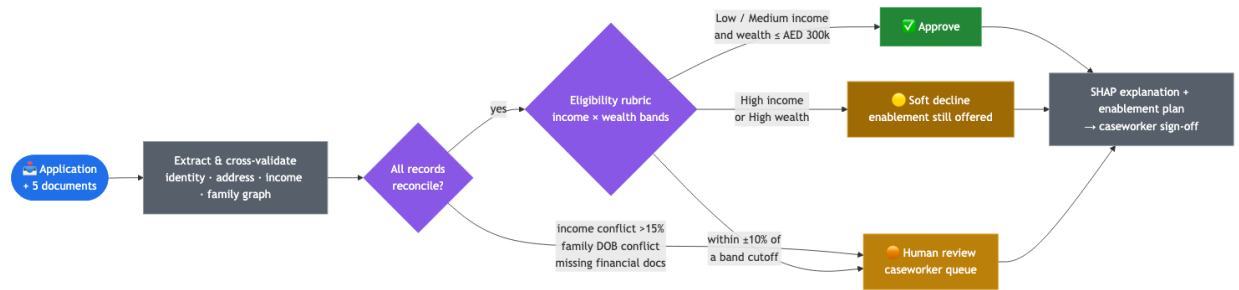


Figure 2 — routing: reconciliation gate first, eligibility rubric second, human review whenever confidence is not justified.

The routing embodies the case study's “up to 99% automated” goal honestly: clear cases finish in the session; the remainder — income conflicts above 15%, family-record conflicts, missing financial evidence, or incomes within 10% of a band cutoff — queue for a caseworker. Outputs are always recommendations for human sign-off, never autonomous rulings.

2.3 Component interactions

Component	Interacts with	Purpose
Streamlit UI	Fast API	Submission, status, chat, dashboards; User and Admin views; EN/AR
Fast API	PostgreSQL, workflow, MongoDB	Persistence, idempotency, audit, versioned /v1 endpoints, basic auth
Extraction agent	MongoDB, Neo4j, vision model	Typed extraction per document; family graph enrichment
Validation agent	Neo4j, LLM	Cross-document consistency; conflict flags that drive routing
Eligibility + decision agents	Classifier, LLM	Rubric signal, probability, SHAP, figure-based rationale
Chat agent	PostgreSQL, Qdrant, LLM	Case-review Q&A grounded in the stored decision record
Citizen registry endpoints	PostgreSQL, MongoDB	Prefill the form and attach a citizen's five documents in one click
Langfuse	all agents	End-to-end traces of every agent/LLM call, self-hosted

3. Technology Choices and Justification

Every tool below is assessed on the five dimensions the case study names: suitability (S), scalability (Sc), maintainability (M), performance (P), and security (Sec). Two decisions frame everything else: models are hosted locally because government PII must not leave the environment, and each data type is handled by the tool built for it rather than forcing one store or one model to do everything.

3.1 Tools per data type

Data type	Tool	Why
Text (bank statement, credit report, resume — PDF)	pypdf + field parsers	Dependency-free text extraction; deterministic parsers make extraction testable against ground truth (100% field accuracy on the seeded set)
Images (Emirates ID, handwritten forms)	minicpm-v:8b vision LLM, Tesseract backstop	Reads structured fields from card images; the classic-OCR backstop keeps extraction alive if the model is unavailable
Tabular (assets/liabilities spreadsheet)	pandas / openpyxl	Direct, reliable Excel parsing; computes net worth without LLM overhead or hallucination risk

3.2 Stack, layer by layer

Layer / choice	Justification across the five dimensions
Reasoning LLM — qwen3.5:9b (MLX build on Apple Silicon)	S: strong bilingual (EN/AR) reasoning for validation, eligibility and chat. Sc: swaps to larger tiers on a GPU profile with no code change. M: one reasoning model for all agents; prompts centralised in one module. P: ~25 s per full application after disabling thinking mode. Sec: fully local — prompts contain applicant data and never leave the machine.
Vision model — minicpm-v:8b (+ Tesseract)	S: accurate structured OCR of ID cards, measured at 93% field accuracy. Sc: replaceable independently of the reasoning model. M: isolated failure domain; a named backstop instead of an improvised one. P: single-image calls of a few seconds. Sec: identity documents are processed locally only.
Embeddings — bge-m3	S: 100+ languages including Arabic, which nomic-style English-first models handle poorly. Sc/M: mature and stable. P: 8K context covers documents without chunking. Sec: vectors stay in the local Qdrant.
Orchestration — LangGraph	S: an explicit state machine is what an auditable government decision pipeline needs — inspectable nodes and edges rather than an autonomous crew. Sc: checkpointing and durable execution paths exist for growth. M: adding an agent is one node and one edge. P: negligible overhead over the LLM calls. Sec: state lives in the application's own stores, not a third-party memory service.
Reasoning frameworks — ReAct + Reflexion	S: ReAct-style reason-then-act across agents; Reflexion self-critique exactly where it pays — the validation agent reviews its own flags and drops within-tolerance noise. P: the extra critique pass is scoped to one agent so latency stays contained.
Classifier — scikit-learn GradientBoosting (+ logistic-regression baseline)	S: small, mixed-type tabular data with non-linear band thresholds is tree-ensemble territory, and government decisions demand explainability (SHAP + feature importances). Sc: retrains in seconds. M: the baseline comparison proves the choice (F1 0.831 vs 0.790) instead of asserting it.

Layer / choice	Justification across the five dimensions
	P: sub-millisecond inference. Sec: trained and served locally; no protected-class features.
Relational — PostgreSQL	S: applications, decisions, the append-only audit log, idempotency keys and the citizen registry are records of legal consequence — ACID and referential integrity matter. Sc: read replicas well beyond prototype needs. M: standard SQL. P: indexed lookups; full-table status aggregation for dashboards. Sec: least-privilege credentials; audit table is insert-only.
Documents — MongoDB + GridFS	S: five attachment types with five shapes; schema-less storage absorbs the variety without migrations. Sc: sharding if volume grows. M: raw bytes, extracted JSON and the registry document store live side by side. P: GridFS streams image binaries. Sec: PII is masked before persistence.
Vectors — Qdrant	S: retrieval over eligibility rules and enablement programs grounds chat answers in real programs. Sc: HNSW indexing as the knowledge base grows. M: two small named collections. P: millisecond search. Sec: self-hosted; no vectors leave the network.
Graph — Neo4j	S: “conflicting family member details” is a relationship question; one Cypher query groups members by name across source documents and flags divergent dates of birth. Sc: traversals stay fast where recursive SQL joins degrade. M: the model is one node type and one relationship. P: conflict check in milliseconds. Sec: read-only credentials for the querying agents.
Observability — Langfuse v2 (self-hosted)	S: traces every agent/LLM call for debugging and audit. Sc/M: v2 deliberately over v3 — v3 adds ClickHouse, Redis and S3-compatible storage, four services this prototype does not need, plus ARM64 friction. Sec: self-hosted because traces contain applicant data inside prompts.
Serving — FastAPI · Front-end — Streamlit	S: async, typed API with generated OpenAPI docs; Streamlit delivers the bilingual demo UI (with User/Admin roles) quickly. Sc: async I/O for concurrent applicants. M: Pydantic schemas catch integration errors early. P: the API adds milliseconds around the model calls. Sec: basic auth on every endpoint; payload-fingerprinted idempotency keys reject key reuse with different data.

4. AI Workflow — Modular Components

The workflow decomposes into the five GenAI agents the case study names — master orchestrator, data extraction, data validation, eligibility check, decision recommendation — plus a chat agent, an ML component, and a registry integration. Each is one module with defined inputs and outputs over a shared typed state, so components can be extended or replaced independently.

4.1 The agents

Agent	Input → output	Notes
Master orchestrator	Application state → trace context, fan-out	LangGraph entry point; owns the execution order and the conditional human-review edge
Extraction	Raw uploads → parsed fields per document	Per-type extractors; mirrors family members into Neo4j tagged by source document
Validation	Parsed documents → conflict flags + resolvable verdict	Six deterministic checks + LLM self-critique; unresolvable conflicts force human review
Eligibility	Parsed data → feature dictionary + rubric signal	Income and wealth bands with a $\pm 10\%$ borderline zone; computes, never decides
Decision	Signal + flags + classifier output → recommendation	Confidence from the model, rationale written from the case's own figures, SHAP attached
Chat	Caseworker question + decision record → grounded answer	Rules select the enablement track; Qdrant retrieval supplies the concrete programs

4.2 Eligibility model

The classifier trains on 200 synthetic applicants generated from the eligibility rubric, with roughly 12% label noise injected deliberately: without it, a model trained on rubric-derived labels would memorise the rubric and score a meaningless 1.0. The noise represents historical caseworker discretion, gives cross-validation results an honest shape, and makes SHAP explanations meaningful. A logistic-regression baseline is trained alongside for comparison.

Metric (5-fold stratified CV)	GradientBoosting	Logistic regression
F1	0.831 $\pm$ 0.033	0.790
Accuracy	0.805 $\pm$ 0.029	0.755
Precision / Recall	0.804 / 0.866	0.762 / 0.821
ROC-AUC	0.861 $\pm$ 0.055	0.829

4.3 Citizen-registry integration

To show how the workflow sits beside existing government systems rather than replacing them, the prototype ships a citizen registry: fifteen profiles in PostgreSQL and five documents per citizen in MongoDB, covering every decision path (clean approvals, income- and wealth-based declines, borderline incomes, an income conflict, a family date-of-birth conflict, an address mismatch). In the UI, “Get User Details” prefills the form from the registry and “Get User Documents” attaches the citizen's documents — the demonstration of integration is itself part of the product.

4.4 Evaluation harness

A dedicated harness (make eval) measures every stage and writes a tabulated report: extraction accuracy per field against registry ground truth, end-to-end routing correctness for all fifteen citizens, classifier cross-validation against the baseline, retrieval quality, and an LLM-as-judge that scores chat answers (groundedness, persona, completeness) and stored rationales (consistency) on a strict 1–5 rubric. The judge earned its keep during development: it caught an Ollama key-value-cache defect that spliced one applicant's facts into another applicant's answer — invisible to ordinary tests — which was then fixed with per-request prompt divergence.

5. Security, Privacy and Fairness

- **Data residency** — every model call is local; Langfuse is self-hosted because traces contain applicant data inside prompts.
- **PII minimisation** — Emirates ID numbers, account numbers and exact income figures are masked before anything is persisted.
- **Access control** — basic auth on the UI and every API endpoint (Caddy fronts public deployments); least-privilege database credentials.
- **Accountability** — an append-only audit trail records every action and decision with the feature values behind it, supporting appeals.
- **Fairness** — nationality, religion and ethnicity are never classifier inputs; a demographic-parity check runs at seed time and publishes approval rates per gender, age band and family-size slice (max disparity 22.6% on the synthetic set — reported, not hidden).
- **Human in the loop** — recommendations require caseworker sign-off; the pipeline routes to a person whenever its own checks undermine confidence.

6. Future Improvements and Integration

6.1 API design considerations

- Replace polling with webhooks or server-sent events for decision-ready notifications to consuming systems.
- Harden the existing idempotency scheme (key + payload fingerprint) into a documented contract for integrators, including retry semantics and error codes.
- Replace demo basic auth with the government identity provider (OAuth2/OIDC single sign-on) and per-role scopes: applicant, caseworker, auditor.

6.2 Data pipeline considerations

- Move ingestion from synchronous upload to an event-driven queue (Kafka or a managed equivalent) so submission spikes buffer instead of blocking.
- Add a batch re-scoring pipeline: when eligibility rules change, re-evaluate open applications and flag decisions that would flip.
- Monitor classifier drift through Langfuse traces and scheduled evaluation runs, with a defined retraining cadence as real labelled decisions accumulate.
- Introduce document virus scanning and size/type quotas at the ingestion edge before anything reaches the extractors.

6.3 Integration with existing systems

- Promote the single-node stores to managed or clustered deployments (replicated PostgreSQL, sharded MongoDB, clustered Qdrant/Neo4j) as volume grows.
- For scale-out inference, move Ollama to a GPU pool behind the same host-boundary security model, or to larger Qwen tiers - a profile change, not a code change.

6.4 Other Improvements

- Upgrade the UI from Streamlit to a more comprehensive framework for production deployment like Next.js or React.
- Create an automated deidentification tool that masks sensitive data and PII and replaces them with LLM generated identical data. The internal link to original name and ID is maintained so that the outcome can be linked to the original person.